

Parallel and Distributed Programming: Assignment 2

Adit Jindal

March 2026

0.1 Introduction

This project focuses on solving a budget-constrained maximum clique problem using parallel computing with MPI. Given a graph where each vertex has an associated profit and cost, the goal is to find a clique (a fully connected subset of vertices) that maximizes total profit while ensuring the total cost does not exceed a given budget.

0.2 Methodology

0.2.1 Over-view

The algorithm implements a parallel Branch-and-Bound (BnB) approach using MPI.

0.2.2 Execution Phases

1. Initialization Each MPI rank independently runs a randomized greedy heuristic to construct feasible cliques. The heuristic uses a perturbed profit-to-cost ratio to diversify solutions across ranks. A global best solution is obtained using `MPI_Allreduce`, and the corresponding clique is broadcast to all ranks. This provides a strong initial upper bound ($P_{\max}$), enabling effective pruning early in the search.

2. Depth-2 Subproblem Generation All ranks deterministically generate the same set of subproblems by expanding the search tree to depth two. Each subproblem corresponds to a partial clique of size one or two along with its candidate set. This avoids communication overhead and ensures consistent partitioning of the search space.

3. Subproblem Ordering Subproblems are sorted in descending order of candidate set size. Larger candidate sets are assumed to correspond to harder subproblems, improving load balance when distributed.

4. Static Load Distribution Subproblems are distributed across ranks using a round-robin scheme:

$$i \bmod P = rank$$

where P is the number of MPI processes. This ensures an even distribution of work without requiring runtime coordination.

5. Parallel Branch-and-Bound Each rank independently solves its assigned subproblems using a recursive BnB procedure. Two bounds are used for pruning:

- **Structural Bound:** Derived via greedy graph coloring, summing the maximum profit from each color class.
- **Knapsack Bound:** A fractional knapsack relaxation based on profit-to-cost ratio.
- **Budget Constraint:** Ensures total cost does not exceed the budget.

0.2.3 Asynchronous Bound Sharing

Sending Updates When a rank finds a better solution, it asynchronously broadcasts the updated $P_{\max}$ to all other ranks using `MPI_Isend`.

Receiving Updates Ranks periodically check for incoming messages using `MPI_Iprobe` and update their local $P_{\max}$ accordingly. This allows improved bounds to propagate without blocking execution.

Communication Cleanup Outstanding non-blocking sends are completed using `MPI_Waitall` before program termination.

0.2.4 Final Aggregation

A global maximum profit is computed using `MPI_Allreduce`. The rank holding the optimal clique broadcasts the solution to all other ranks.

0.2.5 Discussion

Advantages

- Eliminates master-worker bottleneck through decentralized design.
- Strong initial bound improves pruning efficiency.
- Static round-robin distribution provides simple and effective load balancing.
- Asynchronous communication minimizes synchronization overhead.

Limitations

- Depth-2 expansion may lead to $\mathcal{O}(n^2)$ subproblems in dense graphs.
- Static scheduling cannot adapt to runtime imbalance.
- Bounding procedures remain computationally expensive due to sorting and coloring.

0.3 Results and Analysis

0.3.1 Runtime Comparison

Observed trend:

For a large number of neighbours per cluster, Approx Grid KNN > Brute KNN > K-means

For a smaller neighbour/cluster ratio, Brute KNN > K-means > Approx Grid KNN

0.3.2 Accuracy Comparison

Accuracy was measured using:

- Exact match percentage
- Mean Absolute Error (MAE)

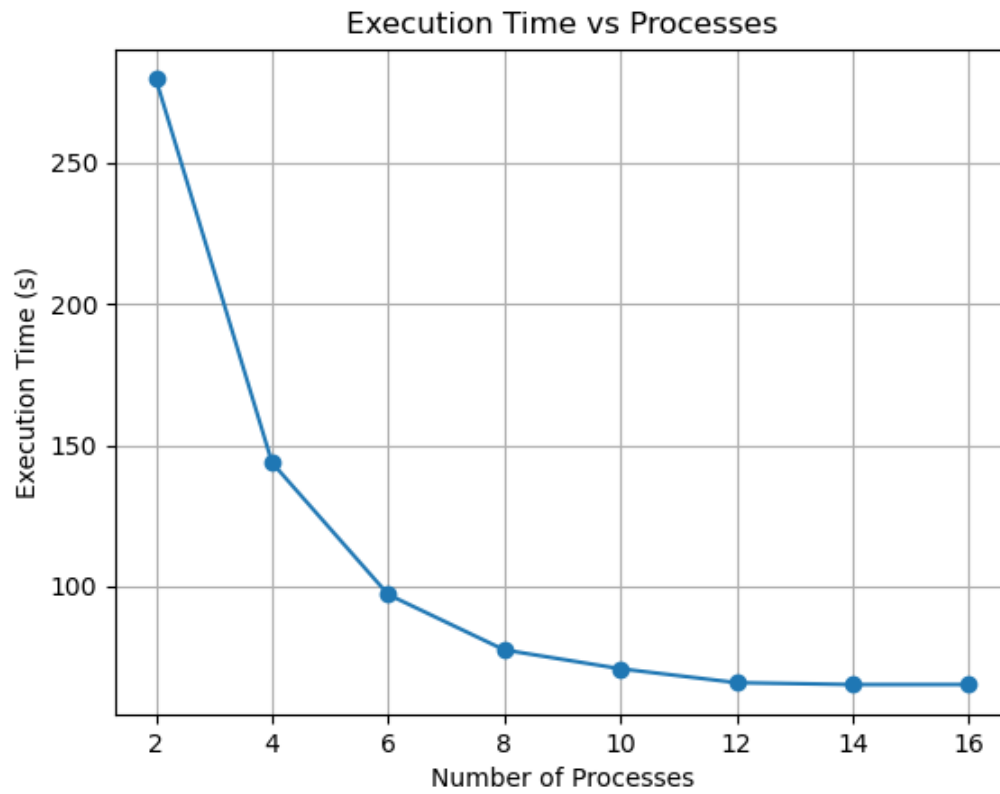


Figure 1: Performance across various runs

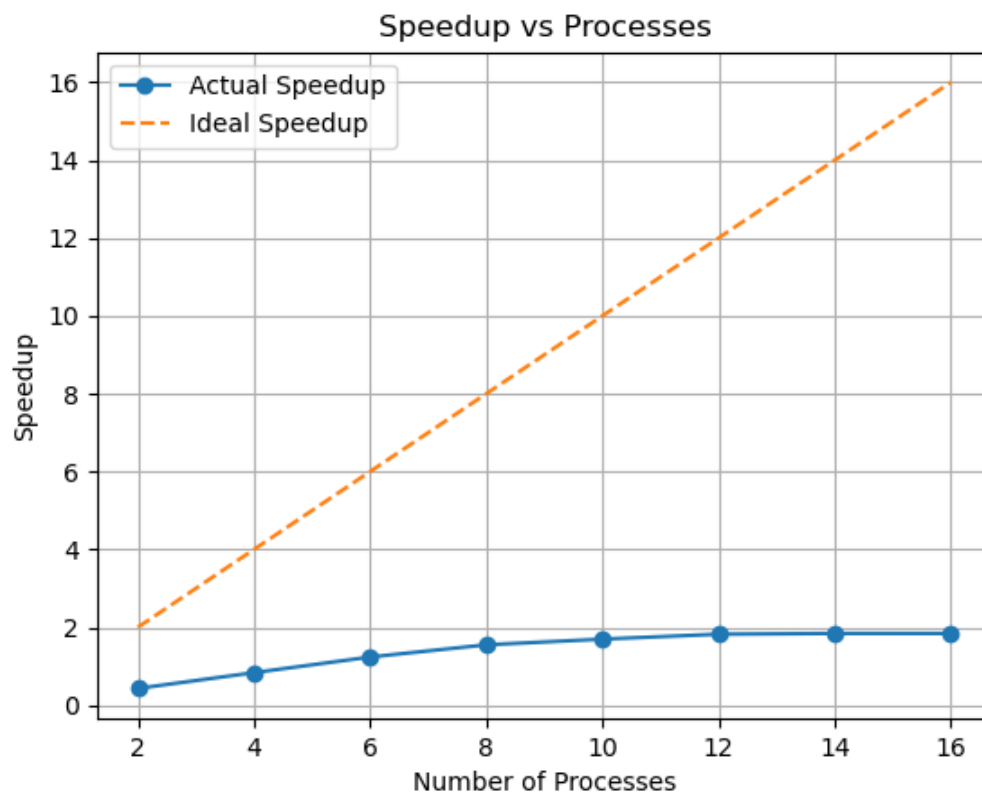


Figure 2: Speedups across various runs (assuming 120s for sequential code)

0.4 Conclusion

We explored multiple strategies for performing local histogram equalization on point clouds. The results demonstrate a clear trade-off between computational efficiency and accuracy. While brute force KNN ensures optimal results, approximate methods such as grid-based hashing and LSH significantly reduce runtime with acceptable loss in quality. K-means clustering provides the fastest solution but at the cost of spatial precision.

Future work may include more advanced approximate nearest neighbor techniques and further optimization using shared memory and warp-level primitives.